

Lab Preliminary Report

Course: Cs 223 Digital
Design

Section: 2

Lab Number: 1

Name: Eralp Yiğit Boz

ID: 22403188

Date: 02.03.2026

PART A

Question 1

4-to-1 Multiplexer With 2-to-1 Multiplexers

A 4-to-1 multiplexer was hierarchically implemented in two stages by interconnecting three 2-to-1 multiplexers, utilizing select lines to systematically route the intermediate and final outputs .

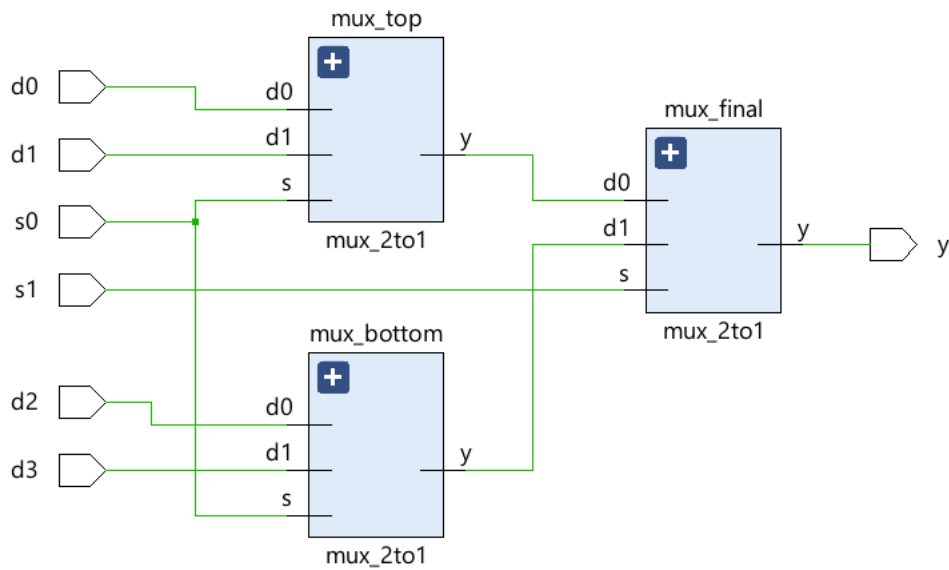


Figure 1.1 4-to-1 Multiplexer Using 2-to-1 Multiplexers

Question2

16-to-1 Multiplexer With 4-to-1

Multiplexers Following a hierarchical modeling approach, a 16-to-1 multiplexer was constructed by cascading five 4-to-1 multiplexers across two stages to evaluate sixteen data inputs .

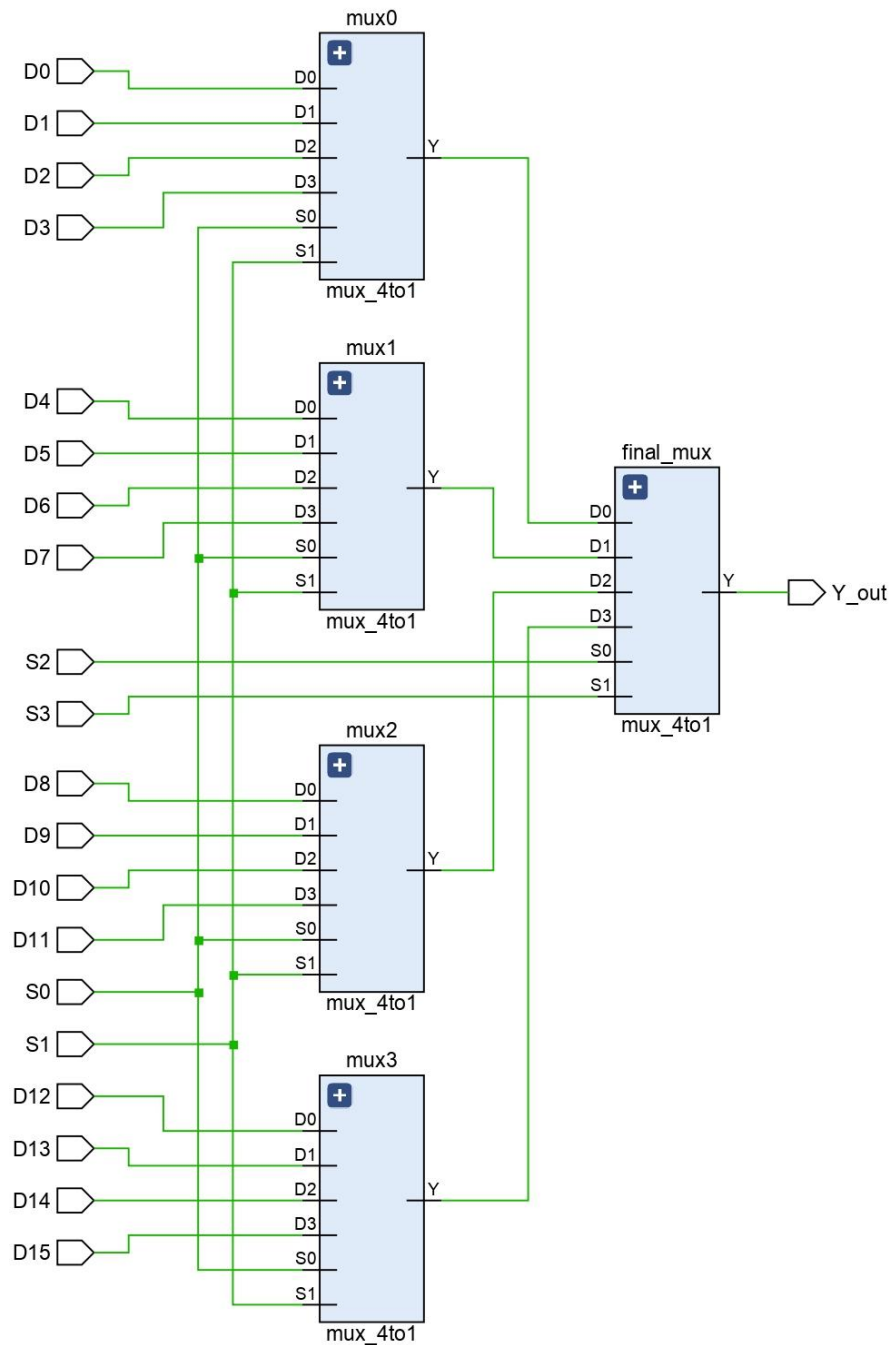


Figure 1.2 16-to-1 Multiplexer Using 4-1 Multiplexers

Question 3

A 2-to-4 decoder equipped with an "Enable" control signal was systematically modeled, detailing its internal logic gate architecture, graphical symbol, and truth table .

Truth Table

Enable (E)	Input A1	Input A0	Output Y3	Output Y2	Output Y1	Output Y0
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

2-to-4 Decoder Symbol

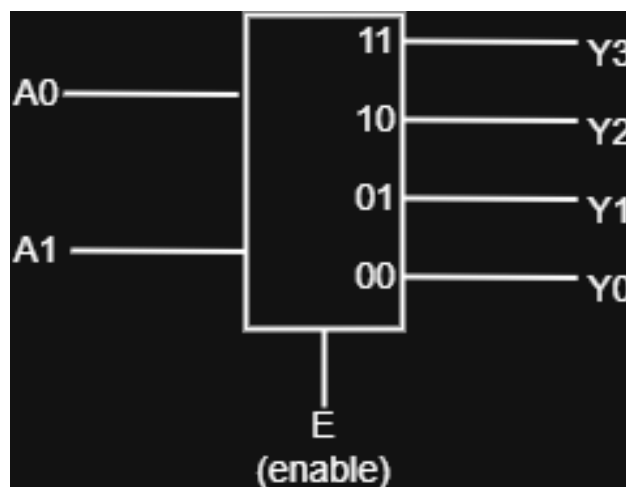


Figure 1.3 2-to-4 Decoder Design

2-to-4 Decoder Logic Diagram

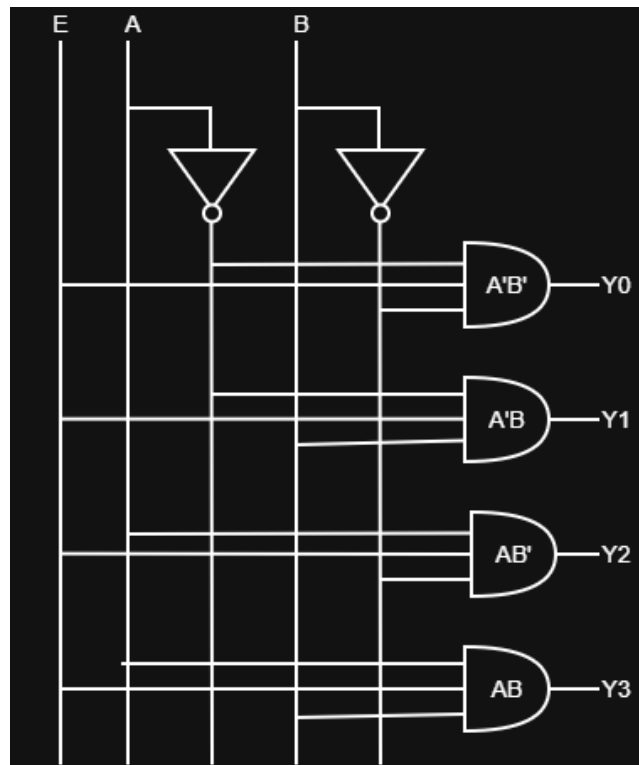


Figure 1.4 3-to-8 Decoder Logic Diagram

Question 4

A 3-to-8 decoder was designed using minimal logic gates by interconnecting two 2-to-4 decoder blocks, utilizing the most significant input bit to selectively activate the appropriate block .

3-to-8 Decoder With 2-to-4 Decoders

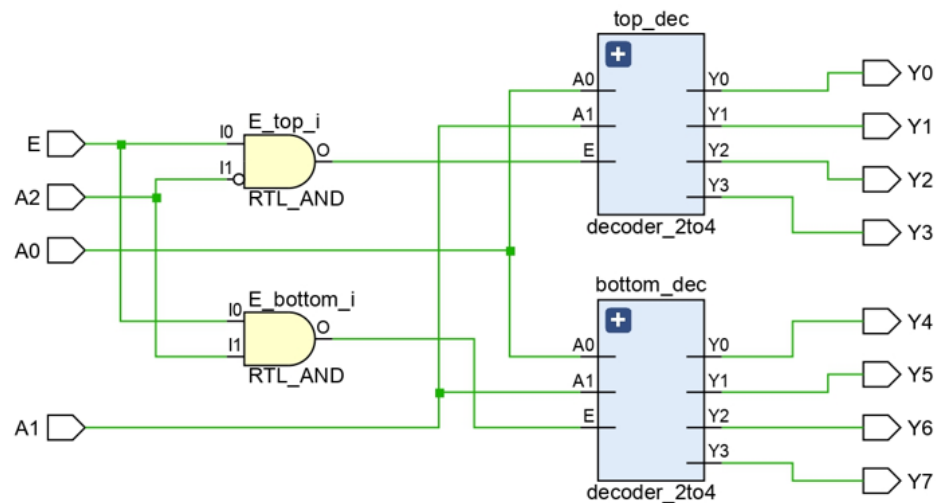


Figure 1.5 3-to-8 Decoder Design

Question 5

SystemVerilog module for 3-to-8 Decoder

```
module decoder_2to4 (
    input logic E,
    input logic A1,
    input logic A0,
    output logic Y0,
    output logic Y1,
    output logic Y2,
    output logic Y3
);
    assign Y0 = E & (~A1) & (~A0);
```

```
    assign Y1 = E & (~A1) & (A0);  
    assign Y2 = E & (A1) & (~A0);  
    assign Y3 = E & (A1) & (A0);  
endmodule  
  
module decoder_3to8 (  
    input logic E,  
    input logic A2,  
    input logic A1,  
    input logic A0,  
    output logic Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7  
);  
    logic E_top;  
    logic E_bottom;  
    assign E_top = E & (~A2);  
    assign E_bottom = E & A2;  
  
    decoder_2to4 top_dec (  
        .E(E_top), .A1(A1), .A0(A0),  
        .Y0(Y0), .Y1(Y1), .Y2(Y2), .Y3(Y3)  
    );  
    decoder_2to4 bottom_dec (  
        .E(E_bottom), .A1(A1), .A0(A0),  
        .Y0(Y4), .Y1(Y5), .Y2(Y6), .Y3(Y7)  
    );  
endmodule
```

Testbench Code

```

module testbench();

    logic E_tb;

    logic A2_tb, A1_tb, A0_tb;

    logic Y0_tb, Y1_tb, Y2_tb, Y3_tb, Y4_tb, Y5_tb, Y6_tb, Y7_tb;

    decoder_3to8 uut (

        .E(E_tb), .A2(A2_tb), .A1(A1_tb), .A0(A0_tb),

        .Y0(Y0_tb), .Y1(Y1_tb), .Y2(Y2_tb), .Y3(Y3_tb),

        .Y4(Y4_tb), .Y5(Y5_tb), .Y6(Y6_tb), .Y7(Y7_tb)

    );

    initial begin

        E_tb = 0; A2_tb = 0; A1_tb = 0; A0_tb = 0; #15;

        E_tb = 1; A2_tb = 0; A1_tb = 0; A0_tb = 0; #15;
        E_tb = 1; A2_tb = 0; A1_tb = 0; A0_tb = 1; #15;
        E_tb = 1; A2_tb = 0; A1_tb = 1; A0_tb = 0; #15;
        E_tb = 1; A2_tb = 0; A1_tb = 1; A0_tb = 1; #15;

        E_tb = 1; A2_tb = 1; A1_tb = 0; A0_tb = 0; #15;
        E_tb = 1; A2_tb = 1; A1_tb = 0; A0_tb = 1; #15;
        E_tb = 1; A2_tb = 1; A1_tb = 1; A0_tb = 0; #15;
        E_tb = 1; A2_tb = 1; A1_tb = 1; A0_tb = 1; #15;

        $finish;

    end

endmodule

```

Part B

4 Bit Full Adder and Subtractor

A 4-bit computational unit capable of both addition and subtraction was implemented using full adders and XOR gates, strictly omitting dedicated full subtractor circuits. A control signal selectively inverts the second operand to execute subtraction seamlessly via two's complement arithmetic.

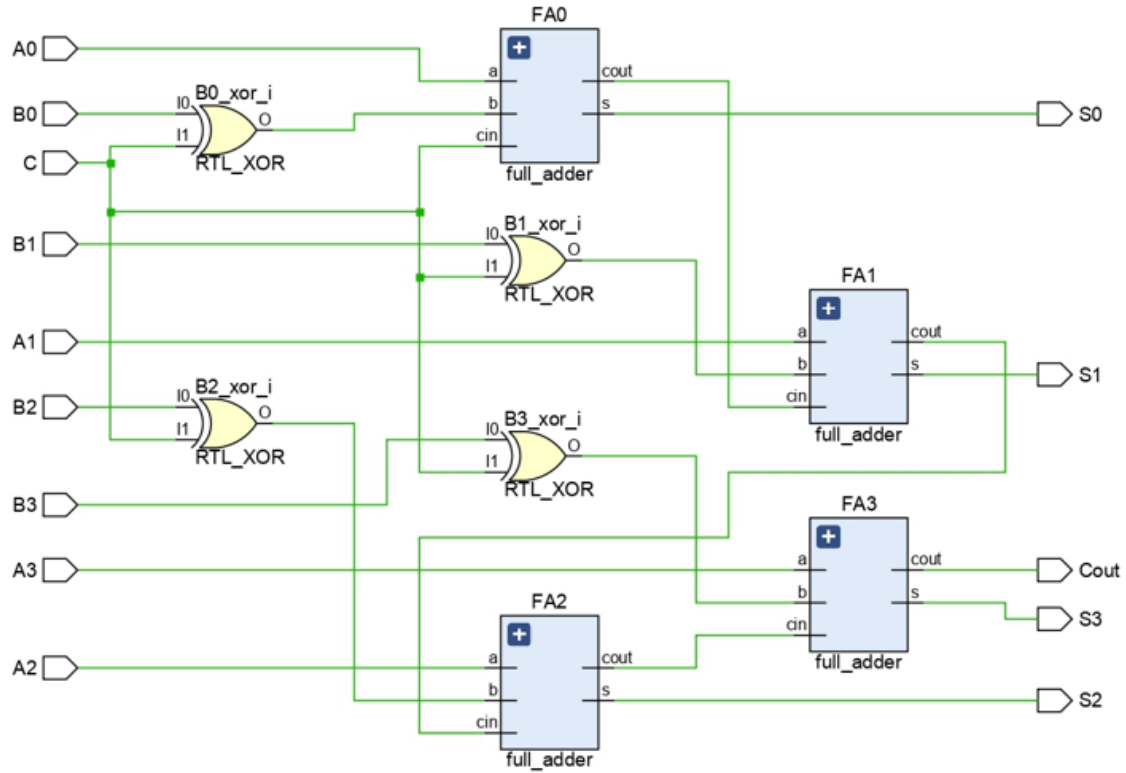


Figure 2.1 Computational Unit Circuit Design

Part C

$$F(A, B, C, D, E) = \sum(0, 1, 2, 3, 4, 10, 11, 16, 17, 18, 19, 25, 28, 29)$$

A five-variable logic function was implemented using exactly one 8-to-1 multiplexer and one 2-to-4 decoder, without any additional logic gates. Input variables were strategically partitioned between the multiplexer selection lines and the decoder inputs to accurately satisfy the specified sum of minterms.

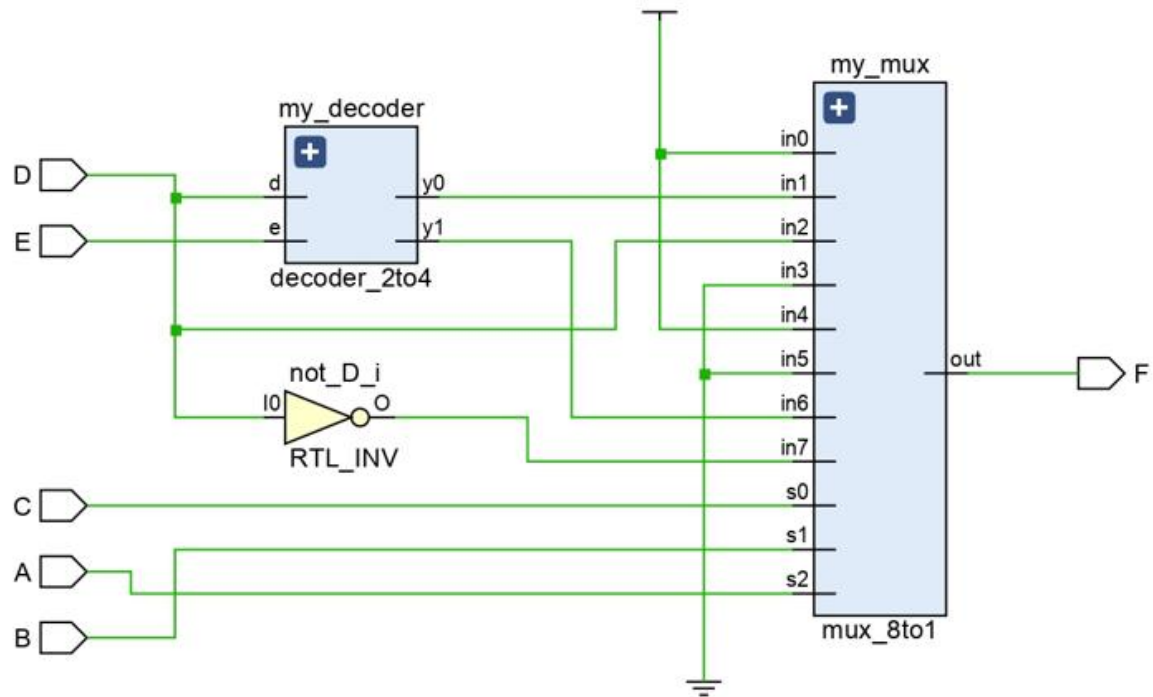


Figure 3.1 Logic Function Design

Systemverilog Code

//2-to-4 Decoder

```
module decoder_2to4 (
    input logic d, e,
    output logic y0, y1, y2, y3
);
    assign y0 = (~d) & (~e);
    assign y1 = (~d) & (e);
    assign y2 = (d) & (~e);
    assign y3 = (d) & (e);
endmodule
```

```
//8-to-1 Multiplexer
```

```
module mux_8to1 (
    input logic in0, in1, in2, in3, in4, in5, in6, in7,
    input logic s2, s1, s0,
    output logic out
);
    assign out = s2 ? (s1 ? (s0 ? in7 : in6) : (s0 ? in5 : in4)) :
                (s1 ? (s0 ? in3 : in2) : (s0 ? in1 : in0));
endmodule
```

```
module logic_function (
    input logic A, B, C, D, E,
    output logic F
);
```

```
    logic y0, y1, y2, y3;
```

```
    logic not_D;
```

```
    assign not_D = ~D;
```

```
    decoder_2to4 my_decoder (
        .d(D), .e(E),
        .y0(y0), .y1(y1), .y2(y2), .y3(y3)
    );
```

```
    mux_8to1 my_mux (
        .in0(1'b1),
```

```
.in1(y0),  
.in2(D),  
.in3(1'b0),  
.in4(1'b1),  
.in5(1'b0),  
.in6(y1),  
.in7(not_D),  
  
.s2(A), .s1(B), .s0(C),  
.out(F)  
);  
  
endmodule
```